

Universidad ORT Uruguay

Facultad de Ingeniería

Machine Learning para sistemas inteligentes

Obligatorio

Documentación

Integrantes:

Francisco Baraibar - 285465

Leandro De Feo - 336280

Juan Diego Fagnoni - 288818

Docentes: Agustina Disiot, Johny Kidd

14 de julio, 2025

Índice

Introducción.....	3
Parte 1.....	3
1.1 Configuración del entorno e importación de librerías.....	3
1.2 Análisis exploratorio inicial.....	4
1.3 Procesamiento de datos.....	5
1.4 Feature Engineering.....	7
1.5 Preparación para modelos.....	7
1.6 Implementación y evaluación de modelos.....	8
1.7 Comparación final de modelos.....	11
1.8 Selección del modelo final.....	12
1.9 Predicciones para Kaggle y Competencia.....	12
Parte 2.....	15
2.1 Configuración del entorno e importación de librerías.....	15
2.2 Carga y verificación de datos.....	15
2.3 Análisis Exploratorio.....	16
2.4 Análisis de campos de texto para NLP.....	17
2.5 Preprocesamiento de texto para NLP.....	19
2.6 Vectorización con TF-IDF.....	20
2.7 Entrenamiento de modelos Baseline.....	22
2.8 Diagnóstico de resultados.....	23
2.9 Optimización Avanzada.....	25
2.10 Evaluación final y selección del modelo.....	26
Uso de inteligencia artificial generativa.....	28

Introducción

Para hacer las tareas pedidas en el obligatorio decidimos crear dos notebooks, uno para la tarea de clasificación binaria para predecir la variable “opinion”, llamado prediccionOpinion y otro para la tarea de NLP para predecir el “main_genre_top10” llamado prediccionGenero.

En cuanto a la organización de la documentación optamos por separar la parte 1 de la parte 2, siguiendo una estructura similar que consta en la explicación de cada uno de los bloques de cada notebook.

Esta decisión hizo que la documentación quedará algo repetitiva, pero nos pareció la forma adecuada sobre todo porque son distintos problemas con distintos enfoques.

Parte 1

1.1 Configuración del entorno e importación de librerías

Dentro de este bloque del notebook establecimos el entorno de trabajo. Hicimos la importación de las librerías necesarias y las dejamos organizadas en base a las etapas del pipeline de la predicción.

Las librerías principales son:

Para análisis de datos: pandas y numpy para la manipulación de datos, junto con matplotlib y seaborn para su visualización durante el análisis exploratorio

Para el modelado y evaluación: se usó la librería scikit-learn para los algoritmos de clasificación, herramientas de validación (train_test_split, GridSearchCV), preprocesamiento y métricas de evaluación (accuracy_score, f1_score).

Warnings: para ignorar los warnings y limpiar los outputs de los bloques.

Google Drive: montaje en Colab para acceder a movies_train.csv y movies_test.csv de forma persistente.

Carga de datos y análisis preliminar

Este bloque carga los datasets de train y test desde Google Drive y muestra la forma inicial (filas y columnas) de nuestros dataframes iniciales.

1.2 Análisis exploratorio inicial

Acá nos aseguramos que los datos de entrenamiento estén correctamente definidos y sean consistentes antes del modelado, eliminando duplicados, verificando los tamaños y asegurando que no queden valores faltantes. Para luego poder pasar al preprocesamiento. Empezamos analizando las primeras 5 filas del set de train para ver el tipo de cada variable (si eran categóricas, numéricas, de texto, fecha, y la etiqueta que nos importa para esa parte: opinion).

Observaciones:

El dataframe tiene 14 columnas, que incluyen:

- Variables numéricas: budget, popularity, revenue, runtime, vote_count.
- Categóricas (texto corto): status, main_genre_top10, original_language.
- Texto libre: keywords, overview, original_title.
- Fecha: release_date.
- Los valores de budget y revenue varían (desde 0 hasta cientos de millones).
- La columna opinion aparece al final y toma valores 0 y 1.

Luego, tomamos información general sobre los datos:

Registros: 3602 — Columnas: 14

Tipos de dato:

- int64: 4 columnas (budget, revenue, vote_count, opinion)
- float64: 2 columnas (popularity, runtime)
- object: 8 columnas

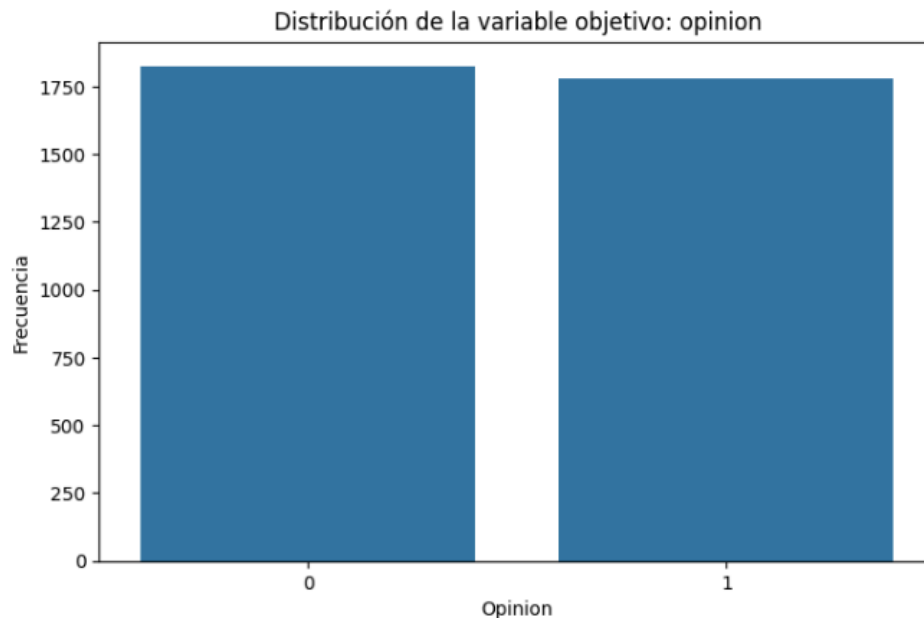
Valores nulos:

- keywords: 3298 non-null: $3602 - 3298 = 304$ nulos
- overview: 3600 non-null: $3602 - 3600 = 2$ nulos
- release_date: 3601 non-null: $3602 - 3601 = 1$ nulo
- director: 3579 non-null: $3602 - 3579 = 23$ nulos
- El resto de columnas aparecen con 3602 non-null, por lo que no hay nulos.

Luego hicimos una tabla de estadísticas descriptivas para las variables numéricas:

- budget y revenue muestran gran skew (muchos valores bajos y unos pocos muy altos).
- runtime se centra alrededor de 100 minutos.
- vote_count tiene medianas bajas (239) pero picos de hasta 14 000.

Al final vimos la distribución de la variable objetivo opinion, con un resultado de 1824 opiniones negativas (0) y 1778 registros con opinión positiva (1). Nos da una proporción de 0: 50.64% - 1: 49.36% y lo representamos visualmente para mayor claridad:



1.3 Procesamiento de datos

En esta parte le aplicamos el mismo preprocesamiento de datos tanto al dataset de train como al de test, haciendo los siguientes cambios:

Conversión de fechas: Transformamos la columna de fecha de lanzamiento a tipo datetime, si habían valores nulos o inválidos los rellenamos con la mediana de las fechas, para asegurarnos que no quede ninguna entrada sin un valor lógico para extraer el año, mes o día de la semana

Tratamiento de variables numéricas: Transformamos cada columna numérica a tipo numérico y sustituimos sus NaN por la mediana correspondiente. Lo hicimos así para preservamos la tendencia central sin que los outliers la desvíen. Las columnas runtime, budget, popularity, revenue y vote_count también se cambiaron a tipo numérico, dejando las celdas vacías como NaN, las cuales como ya mencionamos las cambiamos por la mediana de esa columna. Esta decisión es bastante importante en este caso ya que las columnas como budget o revenue son fuertemente sesgadas, e imputar con la mediana ayuda a mantener la coherencia de la distribución original sin introducir distorsiones.

Estamos introduciendo un poco de ruido falso, pero es un trade off que tenemos que aceptar:

- Es un compromiso razonable: mejor que eliminar toda la fila y perder información.
- La mediana es robusta y no tan sensible a valores extremos como la media.
- Si el porcentaje de valores nulos es bajo, el impacto es mínimo.

No podemos simplemente dejar esos valores como NaN ya que Los modelos de machine learning no pueden trabajar con NaN directamente:

Por ejemplo, RandomForestClassifier, LogisticRegression, GradientBoosting, etc., no aceptan valores faltantes (a diferencia de las técnicas de NLP que ignoran strings vacíos). Si dejamos un valor en NaN, el modelo simplemente no entrena o lanza un error.

Limpieza de texto libre : Cambiamos los null por cadenas vacías ("") en overview, keywords y original_title, para no meter valores falsos. Las columnas overview, keywords y original_title contienen texto libre, por lo que pensamos que debemos procesarlas mediante técnicas de NLP como TF-IDF. Dado que los valores faltantes en este tipo de variables no pueden ser tratados como si fueran categóricos no se debe imputar con la moda, ya que esto nos daría palabras artificiales que no representan el contenido real de la película. Ejemplo, puede pasar que a una película de acción se le asigne la palabra princesa ya que es moda), entonces optamos por reemplazar los valores nulos con una cadena vacía (""). Esta estrategia nos permite mantener la consistencia del dataset sin introducir ruido, ya que los modelos de NLP ignoran estas entradas vacías.

Categóricas de baja cardinalidad: variables con pocas categorías (p. ej. status, original_language, main_genre_top10) rellenamos los nulos con la moda, manteniendo la distribución real sin inventar etiquetas nuevas.

Categórica de alta cardinalidad: en director, que puede tener miles de nombres distintos, rellenar con la moda sesgaría hacia el director más común. Mejor usamos la etiqueta genérica "Unknown", agrupando todos los faltantes sin alterar la frecuencia de los directores reales.

Eliminación de duplicados: Quitamos filas idénticas para asegurar que no entren datos repetidos al modelado.

En variables como original language podemos imputar con la moda porque el conjunto de categorías es chico, la mayoría de los casos coinciden con un idioma dominante entonces el impacto del error es bajo. Por otro lado, para variables como director, donde cada valor representa una persona específica y la cardinalidad es alta, imputar con la moda generaría errores graves. Por eso usamos "Unknown" como categoría neutra.

1.4 Feature Engineering

Comenzamos dropeando las columnas id y title ya que no aportan información predictiva relevante a nuestro modelo.

Luego pasamos a analizar y agrupar los directores:

Calculamos cuántas películas dirige cada persona, obteniendo:

- 1 275 directores con solo una película
- 647 con entre 2 y 9
- 21 con 10 o más

Definimos como umbral 10 películas: todos los directores con menos de 10 títulos los etiquetamos como "Other", mientras que los que alcanzan o superan 10 mantienen su nombre. Al hacer esto, pasamos de 1943 directores distintos a 22 categorías (21 directores frecuentes + "Other").

Guardamos la lista de directores "raros" (1 922 nombres) en rare_directors_list para poder reproducir exactamente la misma agrupación sobre el dataset de test.

Por último, aplicamos One-Hot Encoding en variables categóricas, las variables categóricas (status, original_language, main_genre_top10 y director_grouped) no pueden ser utilizadas directamente ya que la mayoría de modelos de aprendizaje automático necesitan entradas numéricas (no son capaces de procesar texto crudo). Para resolver esto, usamos la técnica de One-Hot-Encoding vista en el curso, esta transforma cada categoría posible en una columna binaria (0 o 1), lo que permite que el modelo interprete la presencia o ausencia de una categoría sin ambigüedad.

Además, utilizamos drop_first=True, esto lo que hace es eliminar una de las n columnas generadas para cada variable, ya que con n-1 columnas podemos deducir que si todas están en 0 el valor de esa variable es el de la columna faltante.

En el caso de variables como status, original_language y main_genre_top10, que contienen pocas categorías distintas, aplicamos One-Hot-Encoding directamente.

Mientras que para las variables de alta cardinalidad realizamos los cambios mencionados anteriormente antes de aplicar esta técnica.

1.5 Preparación para modelos

Aquí nos aseguramos que los datos procesados estén en un formato correcto y divididos apropiadamente para que puedan ser utilizados por los algoritmos de Machine Learning. Primero hicimos la división de los datos en conjuntos de entrenamiento y validación. Para esto comenzamos dividiendo las columnas de las features y la columna de la variable objetivo "opinion" y luego utilizamos train_test_split para la división entrenamiento / validación, los parámetros utilizados fueron:

- `test_size = 0,2` : con esto guardamos el 20% de los datos para la validación
- `stratify = y` : para mantener la misma cantidad de clases en los dos conjuntos
- `random_state = 42` : para fijar la semilla de reproducibilidad y obtener siempre los mismos datos aleatorios

Esta división nos garantiza que ambos conjuntos tengan aproximadamente la misma distribución de clases de la variable objetivo y permite evaluaciones más confiables del rendimiento de cada modelo.

Luego hicimos un procesamiento de los datos para prepararlos finalmente para los modelos, creando un dataset `df_clean` para los algoritmos que requieren datos puramente numéricos. Aplicamos las siguientes técnicas:

- **Conversión de tipos de datos:** columnas booleanas del One-Hot Encoding a enteros (0 y 1)
- **Eliminación de columnas:** eliminamos las columnas de texto libre `overview`, `keywords` y `original_title` y la columna `release_date` ya que estas requieren técnicas de NLP que no vamos a utilizar en esta parte.
- Definimos `num_cols_to_scale` con las variables numéricas `budget`, `revenue`, `runtime`, `vote_count`, `popularity` que necesitarán normalización en modelos sensibles a la escala

Finalmente hacemos una visualización del dataset `df_clean` final donde vemos que se borren bien las columnas, que los tipos de datos son solo enteros y float, que no haya nulos y que la distribución de la variable objetivo mantenga el balance original.

1.6 Implementación y evaluación de modelos

Para cada modelo se utilizaron principalmente 2 métricas para evaluar los modelos:

- **F1-Score:** hace una media armónica entre Precision y Recall
- **AUC-ROC:** calcula el área bajo la curva ROC y mide como distingue las clases el modelo

El framework que implementamos usa tres estrategias de validación, Holdout, Repeated Holdout y Stratified K-Fold Validation. Los modelos utilizan el mismo sistema de evaluación para garantizar que las comparaciones sean justas, se implementaron las siguientes funciones:

- **evaluate_model(model, X_train, X_test, y_train, y_test):** evalúa el modelo con las métricas y las devuelve junto a la matriz de confusión
- **run_holdout(model, X, y):** Hace una validación holdout clásica con división 80/20 y devuelve un DataFrame con métricas y la matriz de confusión.
- **run_repeated_holdout(model, X, y, repeats=10):** Hace 10 iteraciones para evaluación de estabilidad estadística. Devuelve un summary con la media y la desviación estándar
- **run_cross_validation(model, param_grid, X, y):** Hace CV con StratifiedKFold 5-fold. Utiliza GridSearchCV para la optimización de hiperparametros basado en F1-score como métrica principal y devuelve el mejor modelo con sus resultados y los valores de los hiperparametros óptimos

Implementamos los siguientes modelos para la predicción de la variable objetivo opinion:

Regresión logística

Implementamos un modelo de regresión logística como baseline. Construimos un pipeline con StandardScaler() para normalización y regresión logística sin penalización (*penalty=None*).

Evaluamos el modelo con holdout y repeated holdout y estos fueron los resultados principales:

- **F1-Score Holdout:** 0.716
- **AUC-ROC Holdout:** 0.806
- **F1-score Repeated Holdout:** 0.698
- **AUC-ROC Repeated Holdout:** 0.786

Implementamos otra versión regularizada con *penalty='l2'* y optimizando el parámetro C con GridSearchCV. Se muestran los mejores parámetros encontrados y las métricas de cross-validation.

Los resultados principales fueron:

- **F1-score Cross Validation:** 0.708
- **AUC-ROC Cross Validation:** 0.795
- **Mejor configuración:** C=10, *penalty='l2'*

Árbol de decisión

Implementamos un modelo de árbol de decisión utilizando DecisionTreeClassifier. Se evaluó primero la versión con parámetros por defecto sin profundidad máxima para permitir que el árbol crezca sin restricciones. Evaluamos el modelo con holdout y repeated holdout y estos fueron los valores principales:

- **F1-Score Holdout:** 0.669
- **AUC-ROC Holdout:** 0.675
- **F1-Score Repeated Holdout:** 0.658
- **AUC-ROC Repeated Holdout:** 0.661

Luego se hizo una optimización de los hiperparámetros `max_depth`, `min_samples_split`, y `criterion` con `GridSearchCV`. Estos son los resultados principales:

- **F1-Score Cross Validation:** 0.687
- **AUC-ROC Cross Validation:** 0.759
- **Mejor configuración :** `max_depth=5`, `min_samples_split=20`, `criterion='gini'`

Random Forest Classifier

Implementamos un modelo de Random Forest evaluando primero la versión con parámetros por defecto, con 100 estimadores sin restricción de profundidad y después como en árbol de decisión implementamos otro optimizando los hiperparámetros principales con `GridSearchCV`.

Evaluamos el modelo con valores por defecto con holdout y repeated holdout y estos son los resultados principales:

- **F1-Score Holdout :** 0.734
- **AUC-ROC Holdout :** 0.813
- **F1-Score Repeated Holdout :** 0.711
- **AUC-ROC Repeated Holdout:** 0.798

Y para el modelo optimizado los valores y la mejor configuración son:

- **F1-Score Cross Validation:** 0.726
- **AUC-ROC Cross Validation:** 0.813
- **Mejor configuración:** `n_estimators=200`, `max_depth=None`, `criterion='gini'`, `'min_samples_split': 5`

Gradient Boosting

Implementamos igualmente un modelo como baseline y uno con optimización de hiperparámetros.

El primero usa los parámetros por defecto, 100 estimadores y `max_depth=3`, y una `learning_rate=0.1`. Evaluamos el modelo con holdout y repeated holdout y estos fueron los resultados principales:

- **F1-Score Holdout:** 0.756
- **AUC-ROC Holdout:** 0.836
- **F1-Score Repeated Holdout:** 0.714
- **AUC-ROC Repeated Holdout:** 0.812

Para la versión optimizada ajustamos los parámetros con `GridSearchCV`, estos fueron los resultados principales:

- **F1-score Cross Validation:** 0.729
- **AUC-ROC Cross Validation:** 0.823
- **Mejor configuración:** `n_estimators=200`, `learning_rate=0.1`, `max_depth=3`, `min_samples_split=5`

Redes Neuronales

Implementamos un MLP con un pipeline con StandardScaler para normalizar las variables de entrada y un MLPClassifier con una única capa oculta de 100 neuronas usando la función de activación ReLU y early stopping. Evaluamos el modelo con holdout y repeated holdout y estos fueron los Resultados principales:

- **F1-Score de Holdout:** 0.709
- **F1-Score Repeated Holdout:** 0.685
- **AUC-ROC Holdout:** 0.805
- **AUC-ROC Repeated Holdout:** 0.785

Luego implementamos otra MLP optimizando los hiperparámetros del modelo en la cantidad de hidden layers, el parámetro alpha y la learning rate. Los resultados principales fueron:

- **F1-Score CrossValidation:** 0.703
- **AUC-ROC CrossValidation :** 0.798
- **Mejor configuración:** hidden_layer_sizes = (100,) , alpha = 0.0001 , learning_rate_init = 0.001

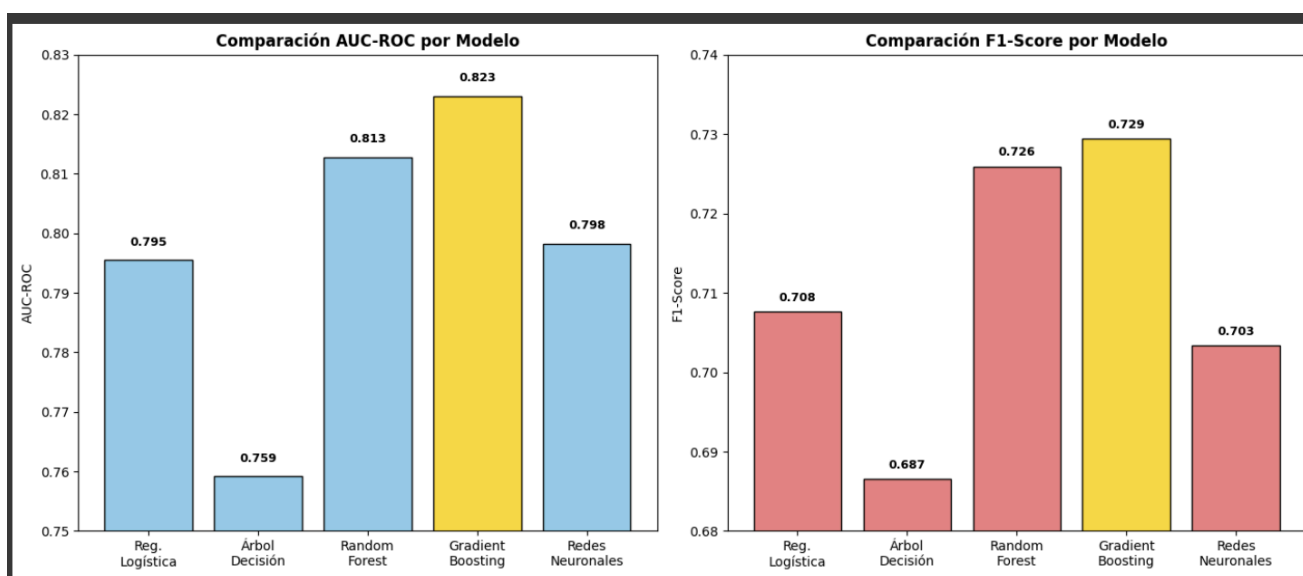
1.7 Comparación final de modelos

En esta parte generamos una tabla de resumen de los 5 modelos probados con sus respectivos resultados de AUC-ROC y F1-Score a modo de comparación final.

Esto nos sirvió para determinar cuál sería nuestro modelo con más potencial para competir en Kaggle.

Además de estas tablas comparativas, nos pareció bueno usar gráficos para comparar los resultados obtenidos en ambas métricas mencionadas.

El gráfico obtenido fue el siguiente:



Por último analizamos las diferencias y concluimos que Gradient Boosting es el que mejor rankea en ambas métricas (AUC = 0.823014 y F1 = 0.729385), igualmente la diferencia con Random Forest no es muy amplia (AUC = 0.812749 y F1 = 0.725936), por eso ambos modelos fueron incluidos en la competencia.

1.8 Selección del modelo final

En este bloque volvemos a justificar porque el modelo elegido es Gradient Boosting, luego elegimos los valores para los hiperparámetros:

- `n_estimators = 200`: Elegimos este número de estimadores (árboles entrenados secuencialmente) ya que conseguimos un buen balance, nos da la capacidad suficiente para aprender patrones sin sobreajuste.
- `learning_rate = 0,1`: Este valor determina cuánto aporta cada nuevo árbol (es el step size), este valor es moderado ya que cada árbol corrige el error de los anteriores de manera gradual, mejorando la generalización.
- `max_depth = 3`: Fijamos este límite como profundidad para cada árbol ya que evitamos que cada árbol aprenda reglas específicas (sobreajuste).
- `min_samples_split = 5`: Es el número mínimo de muestras que debe haber en un nodo para que se considere dividirlo.
- `random_state = 42`: Asegura que la generación de muestras aleatorias y selecciones internas sean idénticas en cada ejecución.

Luego, combinamos el conjunto de entrenamiento limpio en las variables `X_final` (matriz de características) e `y_final` (vector de etiquetas) y entrenamos el modelo Gradient Boosting con estas 3 602 muestras y las 74 características seleccionadas. Después hicimos una última validación manteniendo la misma proporción de antes, (80% entrenamiento y 20% validación), reentrenamos el modelo y generamos predicciones sobre el bloque de validación. Con los resultados, calculamos las métricas clave: una exactitud (accuracy) de 0.7614, precision de 0.7644, recall de 0.7472, F1-Score de 0.7557 y un AUC-ROC de 0.8334, acá podemos ver que los valores se conservan. Por último se imprime la matriz de confusión resultante `[[283 82] [ 90 266]]`, acá podemos ver que el modelo clasificó correctamente 283 instancias negativas y 266 positivas, mientras que dió 82 falsos positivos y 90 falsos negativos. Con estos resultados podemos concluir que Gradient Boosting tiene una cantidad razonable de errores.

1.9 Predicciones para Kaggle y Competencia

En esta última sección del notebook, preparamos el conjunto de test y generamos las predicciones finales que se necesitaban para la competencia en Kaggle.

Para esto, comenzamos aplicando el mismo preprocesamiento usado en el conjunto de

entrenamiento para asegurar la consistencia. Hecho esto entrenamos nuevamente los modelos usando todos los datos disponibles de entrenamiento para maximizar su rendimiento. Luego generamos las predicciones de Gradient Boosting, Random Forest y Regresión Logística optimizada que fueron los modelos que dieron mejores resultados en la métrica AUC-ROC en dicho orden y con resultados muy cercanos.

El motivo por el que subimos todos es porque el resultado al ser probado en publico suele variar, por lo tanto en público podía pasar que Random Forest tuviera mejor rendimiento que Gradient Boosting (aunque no sucedió era necesario probarlo haciendo las submissions correspondientes).

Una vez que hicimos esto, implementamos técnicas de ensemble, donde por ejemplo combinamos Gradient Boosting con Random Forest con distintas ponderaciones para ver si se obtenian resultados mejores a los obtenidos por los modelos individualmente.

Como esto no fue asi y la mejor submission fue submission_gradientBoosting.csv optamos por subirla en el .zip final de entrega. El resultado obtenido por esa submission queda evidenciado en la siguiente imagen:

MLSI Primer Semestre 2025

[Submit Prediction](#)

[Overview](#) [Data](#) [Discussion](#) [Leaderboard](#) [Rules](#) [Team](#) [Submissions](#)

8	Mateo F, Federico G	 	0.81104	2	3d
9	Lucas A y Nahuel P	 	0.81065	5	3d
10	Pereyra-Pinho	 	0.80970	1	4d
11	Rodrigo Soca		0.80469	3	16d
12	RD_JK_JPM		0.79864	13	4d
13	Emilia Caraballo	  	0.79217	29	4h

14 **Fagnoni-Baraibar-DeFeo**



0.78798

5

6d



Your Best Entry!

Your submission scored 0.77921, which is not an improvement of your previous score. Keep trying!

Parte 2

Como comentamos en la introducción, para poder hacer la tarea de predicción de género, hicimos un nuevo notebook en Google Colab para mantener más orden y separación en las tareas pedidas por la letra.

Este notebook tiene una estructura similar al notebook de prediccionOpinion, pero con el uso de técnicas de NLP para predecir el main_genre_top10.

2.1 Configuración del entorno e importación de librerías

En este primer bloque del notebook establecimos el entorno de trabajo. Importamos todas las librerías necesarias para poder hacer esta parte del trabajo y las dejamos organizadas en base a las etapas del pipeline de la predicción.

Las librerías principales son:

Para análisis de datos: importamos pandas y numpy para la manipulación de datos, junto con matplotlib y seaborn para su visualización durante el análisis exploratorio

El **núcleo de esta predicción** usa scikit-learn para algoritmos de clasificación, herramientas de validación, preprocesamiento y métricas de evaluación.

Para **preprocesamiento de texto** incorporamos nltk para el manejo de stopwords y herramientas básicas de Python para limpieza y análisis de texto.

Por último hicimos la conexión a Google Drive al igual que en el primer notebook para poder acceder a los datasets necesarios

2.2 Carga y verificación de datos

En esta sección del código cargamos los dataset desde Google Drive e hicimos una verificación inicial de la estructura de datos para confirmar que contienen la información necesaria para el análisis NLP.

En el análisis determinamos:

Training set tiene 3602 películas y 14 columnas (ya lo habíamos chequeado al hacer la prediccionOpinion) , mientras que test set tiene 1201 películas y 14 columnas, representando aproximadamente una relación 75/25 entre entrenamiento y prueba.

Luego verificamos las columnas críticas para el análisis, los campos de texto para NLP fueron:

- overview: sinopsis de la película
- keywords palabras claves asociadas
- original_title: título original

También determinamos la variable objetivo, en este caso main_genre_top10 y que estuviera completo en todos los registros.

Después de hacer eso analizamos la estructura de datos, mirando las primeras filas para conocer la información que nos daban, como por ejemplo los distintos géneros, contenidos y otros atributos útiles para la predicción.

Por último analizamos la completitud y nos dimos cuenta que la variable objetivo estaba completamente disponible y también los campos de texto principales.

Algo a destacar es que se detectaron algunos valores faltantes en keywords que los manejamos después en el preprocesamiento y también algunos en overview.

2.3 Análisis Exploratorio

En esta parte hicimos el análisis exploratorio del dataset de entrenamiento, enfocándonos en entender bien la estructura de datos, intentando identificar patrones y evaluando la calidad de la información disponible para el modelo de clasificación

Características del dataset

Presupuesto medio \$29.6 millones (rango \$0 - \$380 millones)

Duración promedio 107 minutos(desviación estándar 22.3 minutos)

Popularidad media 21.7 puntos(con alta variabilidad)

Votos promedio: 701 votos por película

Análisis de calidad de datos

Valores faltantes:

-keywords: 304 registros

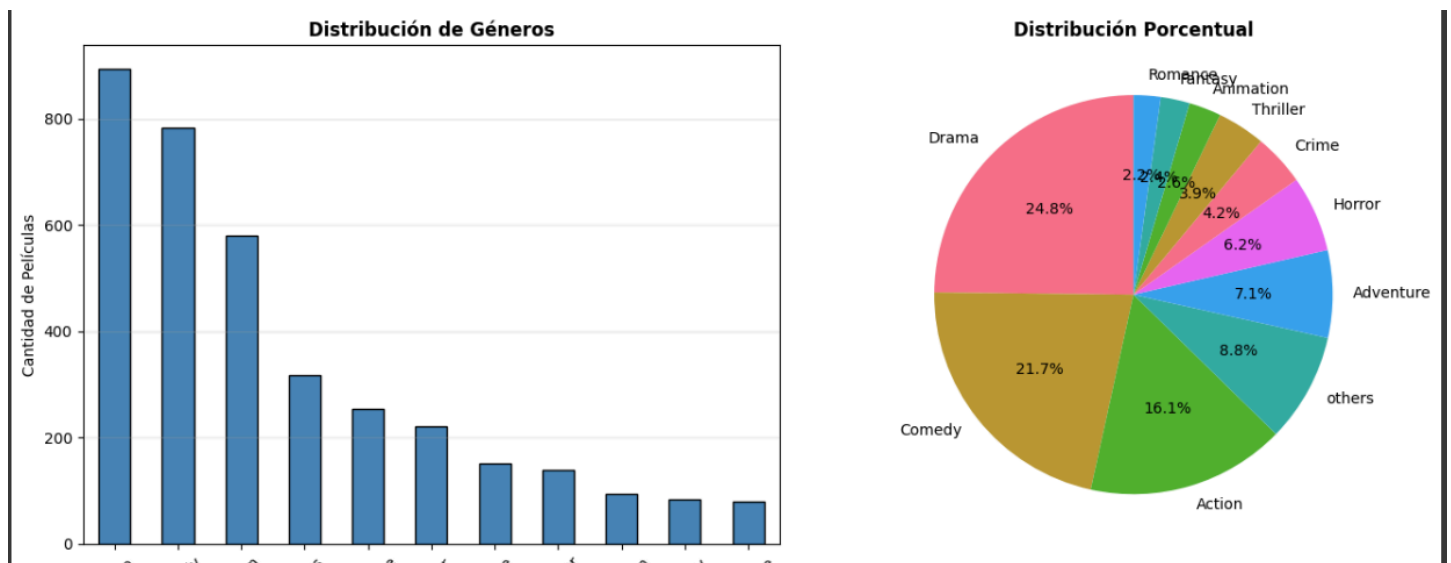
-overview: 2 registros, impacto mínimo

-original_title, completo

Análisis de la variable objetivo

El dataset presenta 11 géneros únicos con una distribución desbalanceada donde los géneros dominantes fueron: Drama, Comedy, Action y los géneros minoritarios fueron: Romance, Fantasy, Animation

Hecho esto nos pareció interesante mostrarlo visualmente como alternativa con los siguientes gráficos



Evaluación del desbalance

Ratio de desbalance 11.3:1, esta proporción entre la clase más frecuente y la menos, no indica un desbalance extremo pero sí tuvimos que considerar estrategias para manejarlo.

Evaluación de entropía

Tiene una entropía de 86,4% este valor indica que el dataset tiene una buena diversidad de clases y que aunque hay desbalance como mencionamos recién, la distribución no está sesgada hacia una única clase, esto favorece al entrenamiento de los modelos.

2.4 Análisis de campos de texto para NLP

En este bloque analizamos los campos de texto a utilizar para la clasificación de géneros. Examinamos la calidad, completitud y características de cada campo.

Overview:

Compleitud: 99,9%

Longitud promedio 305,7 caracteres

Distribución bien balanceada con mediana de 284 caracteres

Representa el campo más valioso para NLP, proporcionando descripciones detalladas de la trama que contienen elementos clave para la identificación de géneros.

Keywords:

Compleitud: 91,6%

Longitud promedio 45,8 caracteres

Distribución compacta y consistente, mediana = 46 caracteres

Representa metadatos limpios que nos proporcionan información semántica clara sobre los elementos temáticos de cada película.

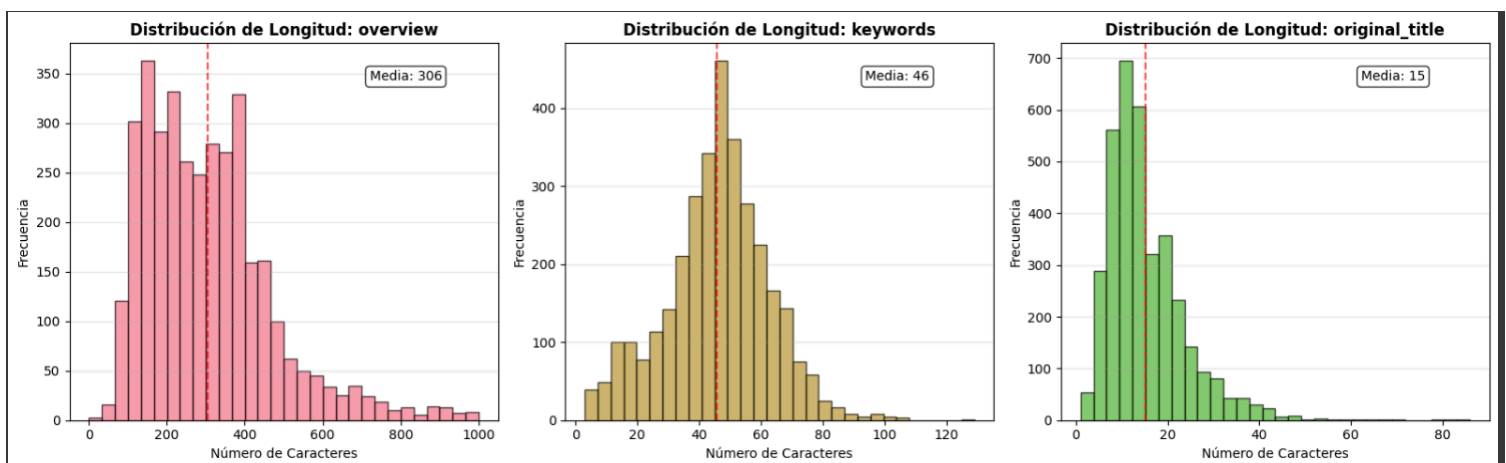
Original Title:

Compleitud: 100%

Longitud promedio 15,2 caracteres

Distribución corta pero universal

En cuanto a la distribución de longitudes también consideramos interesante mostrarlo visualmente con gráficos:



Aquí vemos que:

Overview tiene una distribución normal centrada en 300 caracteres, keywords tiene una distribución concentrada en 45 caracteres y original_title tiene una distribución sesgada hacia títulos más bien cortos

Palabras más frecuentes generales

Identificamos predominancia de stopwords y conectores como: the, and, his, with, confirmando la necesidad de aplicar técnicas de filtrado durante el preprocesamiento

Palabras específicas del género Drama

El análisis por género muestra patrones similares de stopwords, pero con frecuencias proporcionales al tamaño del género

Hecho esto consideramos que por la variabilidad de las longitudes y la presencia de stopwords aplicar TF-IDF iba a ser apropiado para capturar la importancia relativa de términos específicos mientras minimiza el impacto de palabras comunes.

2.5 Preprocesamiento de texto para NLP

Hecho el análisis exploratorio, realizamos el preprocesamiento del texto, necesario para transformar los datos textuales en un formato apropiado para el análisis de NLP. El proceso incluye limpieza, normalización, filtrado de stopwords y combinación de campos textuales.

Funciones aplicadas en el Preprocesamiento

- **clean_text()**: la usamos para normalizar a minúsculas, eliminar caracteres especiales y números, normalizar los espacios en blanco y manejar valores nulos.
- **remove_stopwords()**: para filtrar stopwords estándar en inglés, remover términos específicos del dominio cinematográfico como “movie”, “film”, entre otros y eliminar palabras muy cortas (menos de 2 caracteres)

Proceso de transformación

Etapas del pipeline

- 1) Manejo de valores nulos: conversión de NaN a strings vacíos
- 2) Limpieza básica: normalización y eliminación de caracteres no alfabéticos
- 3) Filtrado de stopwords
- 4) Combinación de campos: concatenación de overview, keywords y original_title

Resultados del preprocesamiento

A continuación dejamos un ejemplo de una transformación, que contiene todos los cambios aplicados en los textos.

Original: "When the Switchblade, the most sophisticated prototype stealth fighter created yet, is stolen from the U.S. government..."

Procesado: "switchblade sophisticated prototype stealth fighter created yet stolen government united states top spies alex scott cal..."

El texto procesado obtuvo una longitud promedio de 263.5 caracteres (se redujó desde 305 caracteres), 36,1 palabras promedio por documento y en cuantos la calidad: 0 textos vacíos, solo 3 documentos con menos de 5 palabras

Reducción de dimensionalidad

Con el preprocesamiento logramos una reducción significativa del ruido residual, manteniendo la información semántica relevante.

2.6 Vectorización con TF-IDF

En este código implementamos la vectorización del texto procesado usando TF-IDF (también podríamos haber usado Bag of Words, la otra técnica vista en el curso), transformando el texto en una representación numérica para que los algoritmos las puedan procesar.

Configuración del vectorizador TF-IDF

Parámetros seleccionados:

- max_feature=5000**, limitamos el vocabulario a los 5000 términos más relevantes
- min_df=3**, excluimos términos que aparecen en menos de 3 documentos
- max_df=0,85** para excluir términos que aparecen en más del 85% de documentos
- ngram_range=(1,2)** incluye unigramas y bigramas para capturar contexto
- sublinear_tf=True** aplica escalado logarítmico para suavizar frecuencias altas

Resultados de la vectorización

Estructura de la matriz:

- Dimensiones: 3602 documentos x 5000 características
- Densidad: 0.0050 (0,5% de valores no cero)
- Sparsity: 99,5% (matriz altamente dispersa)

Vocabulario generado

Con esto logramos generar un vocabulario de 5000 términos que incluye 526 bigramas, los más relevantes de ellos son “advertising executive”, “african american”, “age difference”, esto no incluye como mencionamos ni los terminos muy raros ni los mas comunes.

Codificación de etiquetas

Codificamos los 11 géneros cinematográficos en valores numéricos (0-10), manteniendo la distribución original

División de los datos

Los datos fueron divididos en entrenamiento con un total de 2881 muestras (80%), validación con 721 muestras (20%) y test con 1201 muestras (como conjunto independiente)

La diferencia maxima de 0,00008 entre las distribuciones de entrenamiento y validación nos confirma que la división realizada preserva correctamente las proporciones de cada clase

Hecho esto, ya dejamos los datos preparados para el entrenamiento de los modelos de clasificación con una estructura computacionalmente eficiente.

2.7 Entrenamiento de modelos Baseline

En este bloque de código hicimos el entrenamiento de los 5 algoritmos de clasificación para establecer una línea base de rendimiento y comparar la efectividad de cada enfoque en el problema de clasificación de géneros.

Función de evaluación

Primero implementamos una función estándar que entrena cada modelo y calcula métricas clave

- Accuracy: rendimiento general del modelo
- F1-Score Macro: promedio no ponderado del F1 por clase
- F1-Score Weighted: F1 ponderado por tamaño de clase
- Precision y Recall: como métricas complementarias por clase

Resultados obtenidos por cada algoritmo

1) Regresión Logística

Train Accuracy: 100%

Val accuracy: 38,56%

F1-Macro: 0,2401

Observación: overfitting severo (diferencia del 61,44%)

2) Arbol de decision

Train Accuracy: 34,36%

Val accuracy: 30,51%

F1-Macro: 0,1087

Observación: rendimiento bajo pero sin overfitting

3) Random Forest

Train Accuracy: 47,34%

Val accuracy: 34,67%

F1-Macro: 0,1122

Observación: tiene overfitting ligero y un rendimiento moderado

4) Gradient Boosting

Train Accuracy: 99,31%

Val accuracy: 34,95%

F1-Macro: 0,2190

Observación: tiene overfitting extremo (diferencia del 64,36%)

5) Red Neuronal (MLP)

Train Accuracy: 100%

Val accuracy: 40,50%

F1-Macro: 0,2647

Observación: es el modelo con mejor rendimiento pero tiene overfitting

Luego de obtener estos resultados concluimos que los modelos más complejos muestran overfitting, memorizando los datos de entrenamiento y no logrando generalizar adecuadamente.

Por lo tanto en el siguiente bloque hicimos un diagnóstico para luego poder hacer una optimización de hiperparametros y regularización.

2.8 Diagnóstico de resultados

Una vez que hicimos el entrenamiento de los modelos baseline hicimos este bloque en forma de diagnóstico para hacer un análisis detallado de los resultados obtenidos por dichos modelos. El principal problema que detectamos fue el overfitting en tres de los cinco modelos.

Comparativa General de Modelos

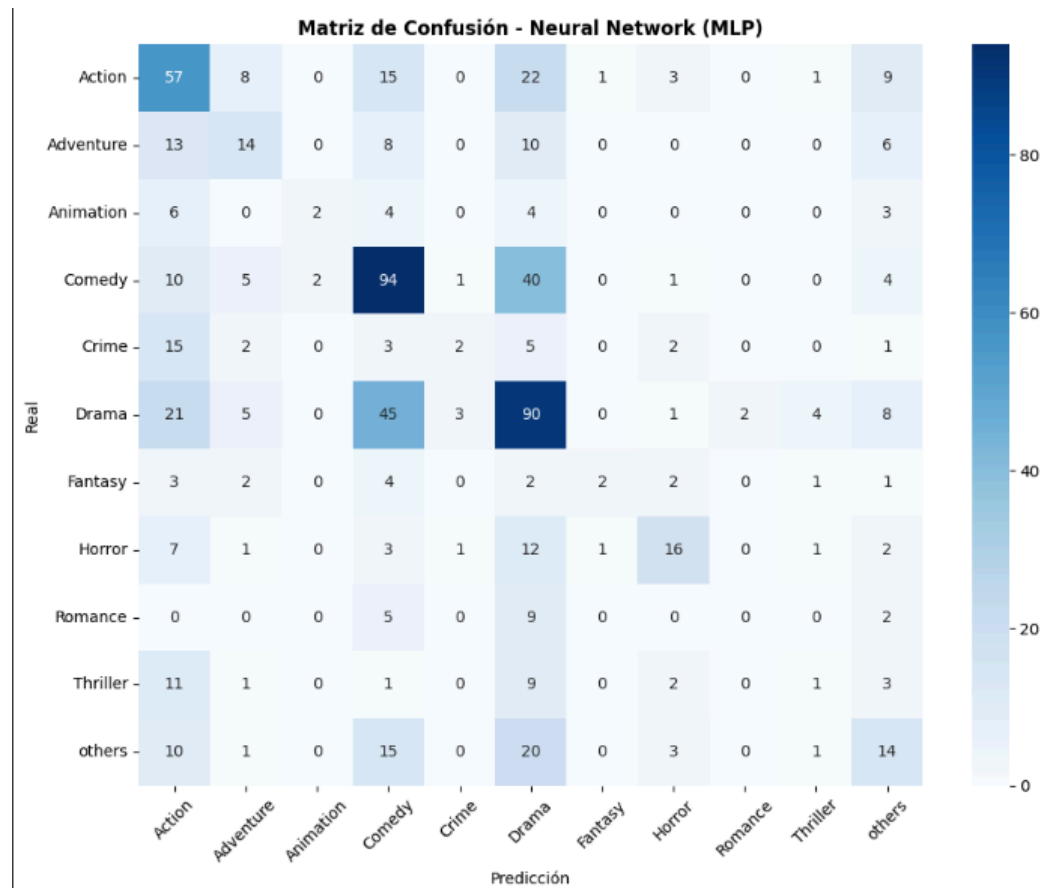
Comenzamos haciendo una comparativa de los resultados obtenidos en el bloque anterior, para esto mostramos un ranking por accuracy de validación donde el orden por rendimientos obtenido fue el siguiente:

- 1) MLP
- 2) Logistic Regression
- 3) Gradient Boosting

4) Random Forest

5) Decision Tree

Como mencionamos anteriormente tanto Gradient Boosting como Logistic Regresion y MLP están memorizando los datos de entrenamiento. Para esto decidimos mostrar la matriz de confusión de MLP que revela ciertos patrones importantes:



Acá pudimos observar:

- **Clases bien clasificadas:** comedy, drama, action
- **Clases Problemáticas:** romance, crime, animation

Confusiones principales

1) drama - comedy: $45 + 40 = 85$ confusiones mutuas

2) Action > Drama: 22 errores (géneros conceptualmente próximos)

3) Others > Drama/Comedy: 35 errores (categoría ambigua)

Problemas identificados en el diagnóstico

- Se necesitan regularizar los modelos complejos
- Las clases minoritarias necesitan técnicas de balanceo
- Géneros cinematográficos naturalmente se superponen

Todos estos puntos buscamos mejorarlos en el siguiente bloque que consta en una optimización avanzada.

2.9 Optimización Avanzada

En este bloque implementamos varias técnicas de optimización buscando mejorar el rendimiento de los modelos mediante Grid Search, ajuste de hiperparámetros y optimización de la vectorización de texto

Los principales procesos que implementamos fueron:

1) Re-vectorización Optimizada

- Reducimos características de 5000 a 2000
- Optimizamos los parámetros TF-IDF: min_df = 5, max_df = 0,7, ngram_range = (1,2)
- Mejoramos la densidad de la matriz y reducimos ruido

2) Grid Search para Regresión Logística

- Explora parámetros: C, class_weight, solver
- Configuración óptima = C = 0,1, class_weight = 'balanced', solver = 'lbfgs'
- Accuracy: 45,08%
- F1-Macro 31.42%

3) Grid Search para MLP

- Optimiza arquitectura, regularización y tasa de aprendizaje
- Configuración óptima: 100 neuronas, alpha=0,001, lr=0,01
- Accuracy: 47,02%
- F1-Macro: 26.00%

4) Cross - Validation para Regresión Logística Optimizada

Metodo: 5-fold Stratified Cross-Validation

Evaluamos el modelo de regresión logística con hiperparametros optimizados, ya que obtuvo el mejor F1-Macro entre todos los modelos optimizados,

Los resultados obtenidos de Cross Validation fueron:

F1-Macro promedio: 0,3000

Desviación estándar +- 0.0064

Lo que indica una alta estabilidad del modelo, por lo tanto seleccionamos como modelo final Regresión Logística con Grid Search, por tener mejor balance entre F1-Macro y estabilidad.

Mejoras vs Modelos Baseline

Tomando como referencia el modelo MLP que fue el de mejor resultado base

- Accuracy: incremento de 40,50% a 45,08%
- F1-Macro: incremento de 26,47% a 31,42%
- Overfitting: se redujo notoriamente de 59,5% a 27,5%, generalizando mucho mejor

Por lo tanto, en cuanto a beneficios técnicos mejoramos el rendimiento predictivo, la generalización y una estabilidad robusta.

2.10 Evaluación final y selección del modelo

Finalmente dedicamos esta última sección del notebook para dar cierre y evaluar la selección del modelo óptimo, basado en distintos criterios y generación de predicciones para el conjunto de test.

Análisis Comparativo

Como vimos en los bloques anteriores, evaluamos un total de 7 configuraciones, divididas en dos categorías.

5 modelos originales con parámetros por defecto y 2 modelos optimizados con Grid Search, en nuestro caso Regresión Logística y MLP.

Mejores configuraciones por algoritmos

Identificamos el mejor rendimiento por familia de algoritmos, donde los modelos sometidos a Grid Search mostraron grandes mejoras en comparación a los 5 modelos con parámetros por defecto

Selección final del modelo

Para determinar esto los criterios de selección que aplicamos fueron

- 1) F1-Score Macro: priorizando el balance entre clases
- 2) Accuracy de validación: rendimiento general
- 3) Control del overfitting: por capacidad de generalización
- 4) Estabilidad de Cross validation: robustez estadísticas

Finalmente el modelo elegido como se mencionó en la parte anterior fue Regresión Logística con Grid Search.

Métricas de rendimiento del mejor modelo

En la parte anterior las mencionamos pero decidimos ponerlas aca como cierre final

Accuracy: 45.08%

F1-Macro: 31.42%

F1-Weighted: 44.03%

Overfitting: 27,50%

En cuanto a la justificación técnica de esta elección, es el modelo con mejor balance precision-recall, tiene un overfitting controlado, su estabilidad fue validada, es apropiado para datos textuales de alta dimensionalidad.

Generación de predicciones en Test Set

Aplicación del Modelo Final

Dataset de test: 1201 películas procesadas

Confianza promedio: 11,6%

Distribución de Predicciones

Drama: 303 películas, género más predicho

Comedy: 269 películas

Action: 210 películas

Others: 109 películas, categoría residual

El resto de las predicciones tienen resultados similares entre sí, aproximadamente 60 películas.

Esta distribución refleja tanto los sesgos del dataset de entrenamiento como las tendencias naturales del cine, donde Drama y Comedy suelen dominar en cuanto a producción.

Analisis de confianza

La confianza promedio de 11,6% es baja, pero es lo esperado en una clasificación multiclase de 11 categorías. Sin embargo presenta una mejora notoria en comparación al 9,09% del azar, siendo una mejora significativa para este tipo de problemas complejos de NLP.

Además dicho rango de confianza es parejo para todas las categorías, esto nos indica que tiene un comportamiento consistente y que no genera predicciones con confianzas excesivamente altas, siendo lo más adecuado para problemas de este tipo.

Uso de inteligencia artificial generativa

Nos pareció correcto dedicar una última parte de la documentación para aclarar y evidenciar que se utilizó Inteligencia Artificial para algunos aspectos del desarrollo del obligatorio.

El uso que le dimos fue principalmente para dos aspectos:

1) Para dar mayor organización a los notebooks, agregando títulos prolijos y bien definidos, respuestas bien seccionadas en los resultados de cada bloque y con comentarios lo más explicativos posibles, ya que tenemos bloques que abarcan varios puntos de procesos, como por ejemplo en preprocesamientos donde se hacen varios pasos.

2) También le dimos para poder documentar de la forma más detallada y completa posible, abarcando todos los resultados de cada sección de los notebooks e intentando mantener una estructura prolija.

Queremos aclarar que todo el texto de la documentación fue escrito por nosotros y que el objetivo de esto fue dar mejor estructura y mayor completitud, evitando que nos olvidáramos de algún resultado obtenido o decisión tomada.

Es por esto que a lo largo de la documentación se ven resultados muy detallados y específicos, como porcentajes o resultados de métricas.